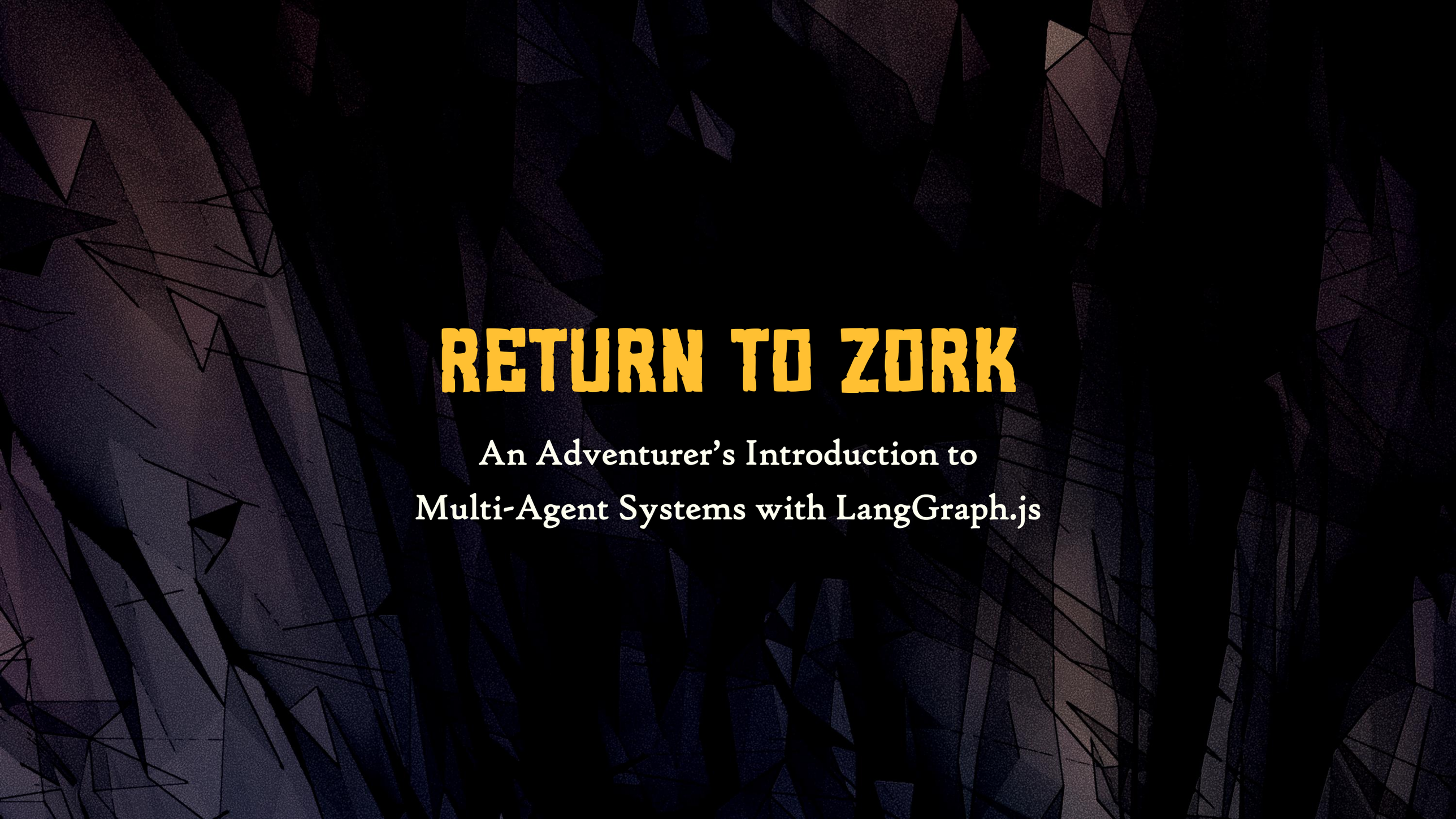

RETURN TO ZORK
An Adventurer's Introduction to
Multi-Agent Systems with LangGraph.js



Guy Royse

youtube.com/@guyroyse

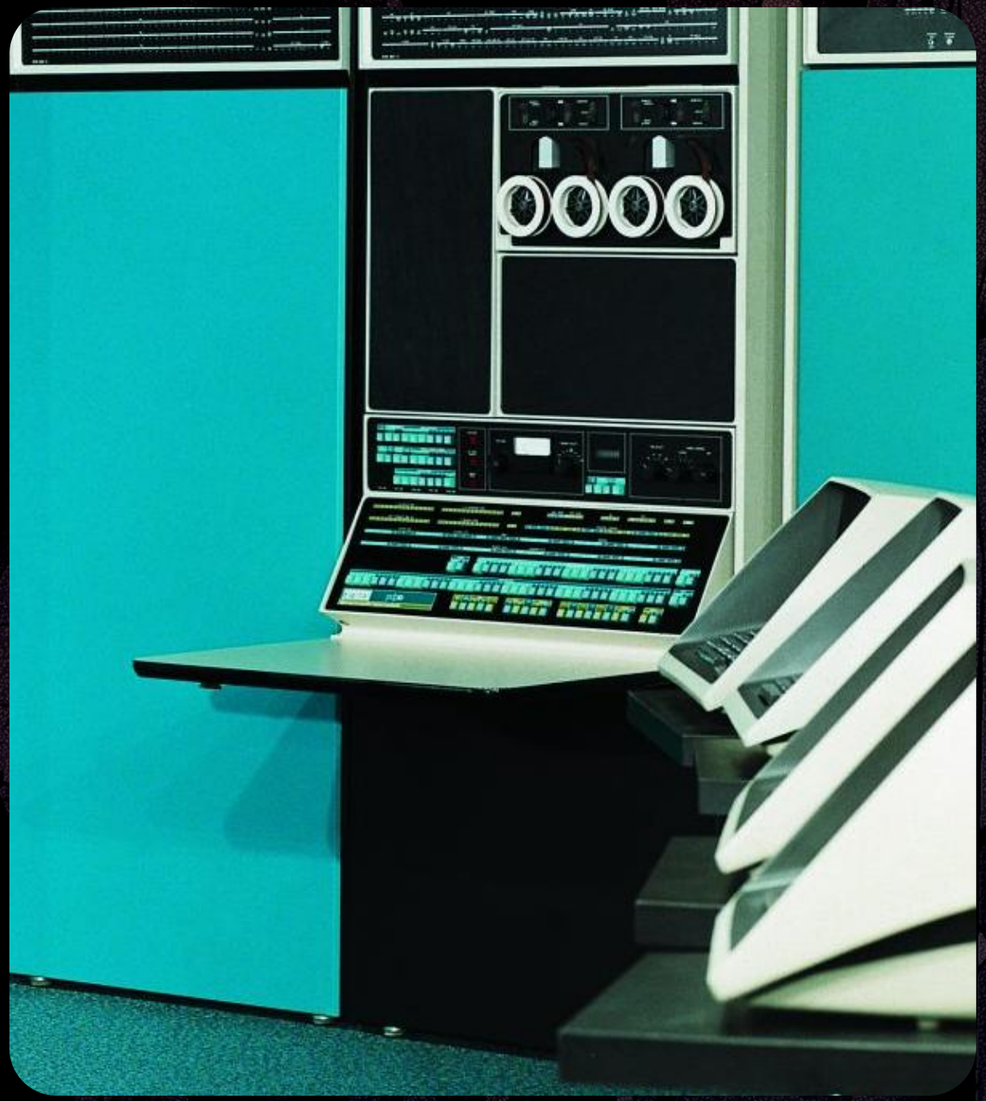
github.com/guyroyse

@guy.dev

@guyroyse

guy.dev


ZOOK




CTRL
RUN STOP
SHIFT LOCK
SHIFT

West of House Score: 0 Moves: 0

>Look
West of House
You are standing in an open field west of a white house, with a boarded front door.
There is a small mailbox here.

>Look in mailbox
The small mailbox is closed.

>Open mailbox
Opening the small mailbox reveals a leaflet.

>Take leaflet
Taken.

>Look at leaflet
"WELCOME TO ZORK!

ZORK is a game of adventure, danger, and low cunning. In it you will explore some of the most amazing territory ever seen by mortals. No computer should be without one!"

>_

```
"ZORK1 for
            Zork I: The Great Underground Empire
   (c) Copyright 1983 Infocom, Inc.  All Rights Reserved."

<VERSION ZIP>

<SETG ZORK-NUMBER 1>

<SET REDEFINE T>

<OR <GASSIGNED? ZILCH>
    <SETG WBREAKS <STRING !\" !,WBREAKS>>>

<PRINC "Renovated ZORK I: The Great Underground Empire
">

<FREQUENT-WORDS?>

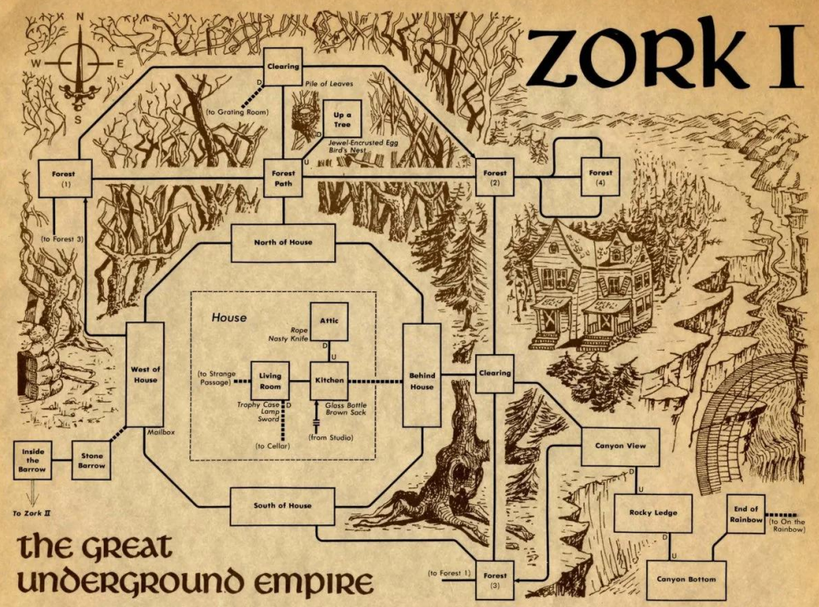
<INSERT-FILE "GMACROS" T>
<INSERT-FILE "GSYNTAX" T>
<INSERT-FILE "1DUNGEON" T>
<INSERT-FILE "GGLOBALS" T>

<PROPDEF SIZE 5>
...
```

zork1.zil

ZORK I

the GREAT UNDERGROUND EMPIRE

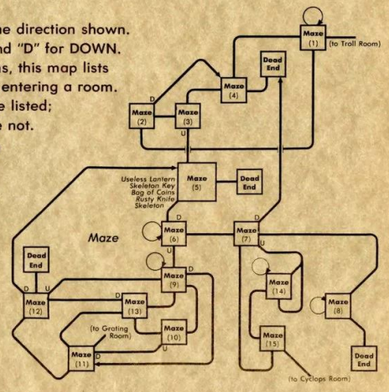


Map designed by D. Ardito and S. Meretzky
© Copyright 1982 Zork Users Group

Legend

- **Normal passageway**
- **One-way passageway**
- **Passageway requiring special equipment or problem-solving**
- **Narrow passageway (baggage limit)**
- **Passageway returning to room of origin**

Notes: All horizontal passages leave the room in the direction shown. Vertical passages are labelled "U" for UP and "D" for DOWN. To avoid unnecessarily giving away problems, this map lists only those objects immediately visible upon entering a room. Only objects which can be taken or used are listed; objects which are merely part of a room are not. Where more than one direction leads to the same place, all are not necessarily shown.



For information and price lists for other games and products write:

ZORK USERS GROUP
P.O. BOX 20923
MILWAUKEE, WI 53220-0923

Zork is a trademark of Infocom Inc. The Zork Users Group is an independent group not affiliated with Infocom Inc., the producers of Zork software.

ZORK I

the GREAT UNDERGROUND EMPIRE



GAME STATE



"You are facing the north side of a white house. There is no door here, and all the windows are boarded up. To the north a narrow path winds through the trees."

"You are standing in an open field west of a white house, with a boarded front door."

"This is a forest, with trees in all directions. To the east, there appears to be sunlight."

closed: false

MAIN GAME LOOP

Describe

Parse

Process

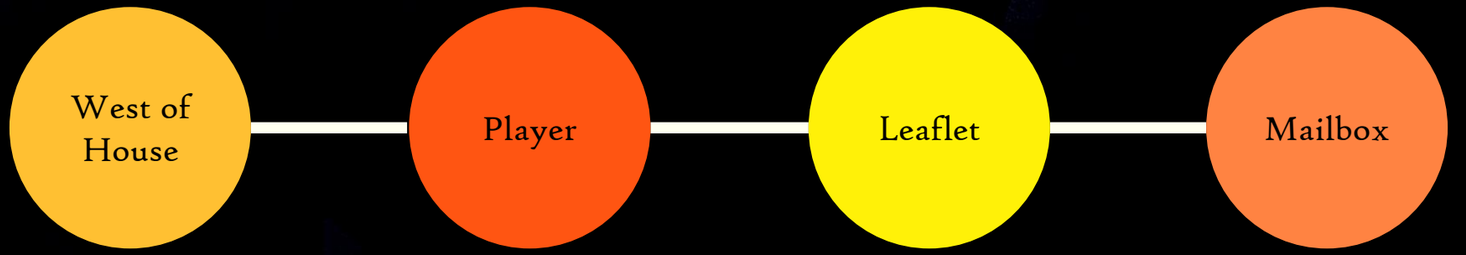
Timers

You are standing in an open field west of a white house, with a boarded front door.

LOOK
OPEN THE MAILBOX
LOOK AT THE MAILBOX
TAKE THE LEAFLET FROM THE MAILBOX

LOOK
OPEN MAILBOX
LOOK MAILBOX
TAKE LEAFLET MAILBOX





closed: false


COMMANDS
ARBITER
RESPONSES

MAIN GAME LOOP

Describe

You are standing in a field next to a white house. The front door is boarded shut. There is a rusty mailbox here leaning at an angle.

Parse

I open the mailbox. Is there anything in it?

Process

Remove
"Rusted Shut"

Add
"Open", "Rusty"

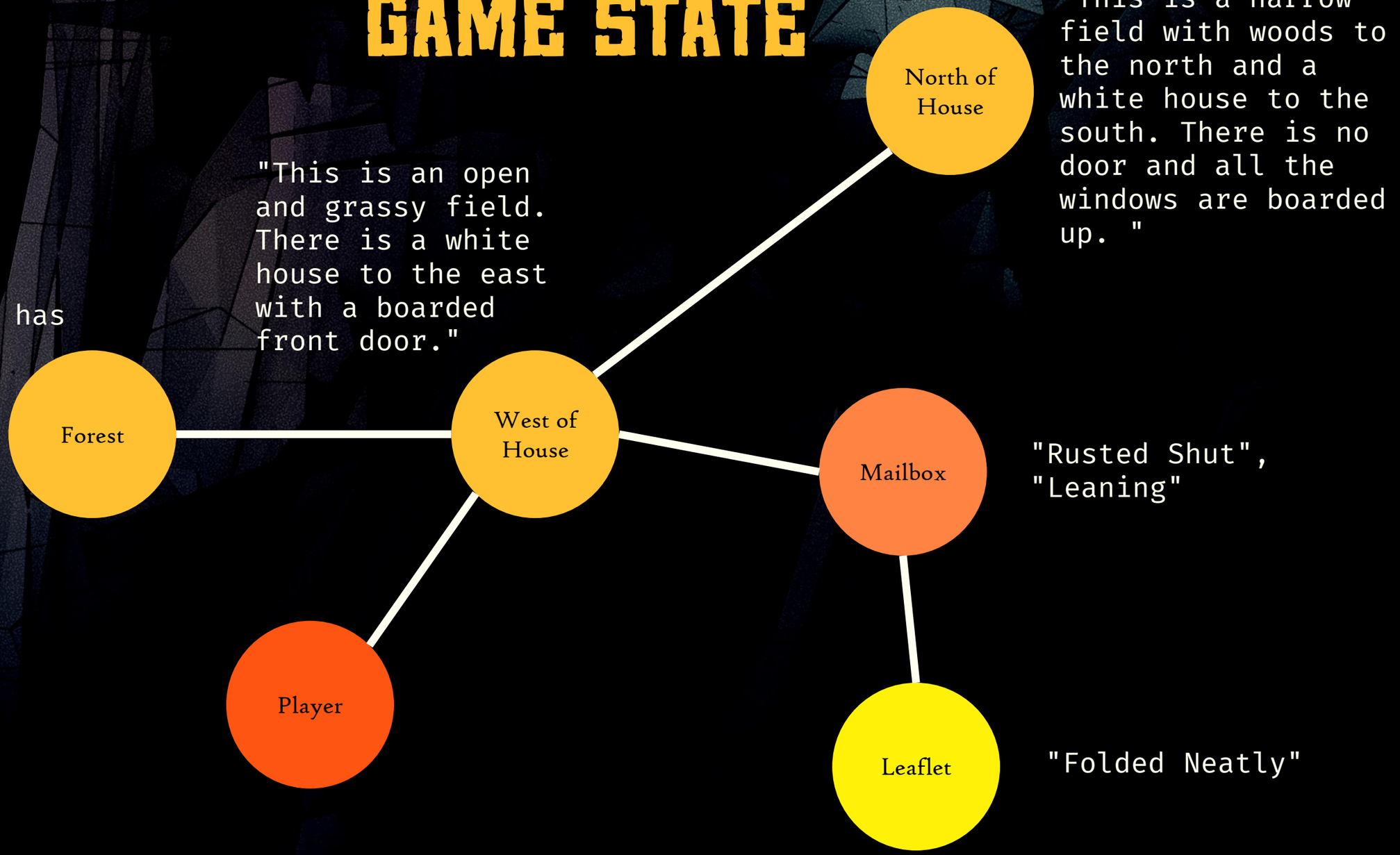
Timers



Player — West of House — Mailbox

"Leaning", "Open", "Rusty"

GAME STATE


North of House
"This is a narrow field with woods to the north and a white house to the south. There is no door and all the windows are boarded up. "
"This is an open and grassy field. There is a white house to the east with a boarded front door."
"This forest has trees in all directions."
Forest
West of House
Mailbox
"Rusted Shut", "Leaning"
Player
Leaflet
"Folded Neatly"


HIC HABITAT
GENIVS LOCI

GENII SINGULORUM LOCORUM

Forest



West of House



North of House



Mailbox



Leaflet



WHAT'S AN AGENT?



LLM

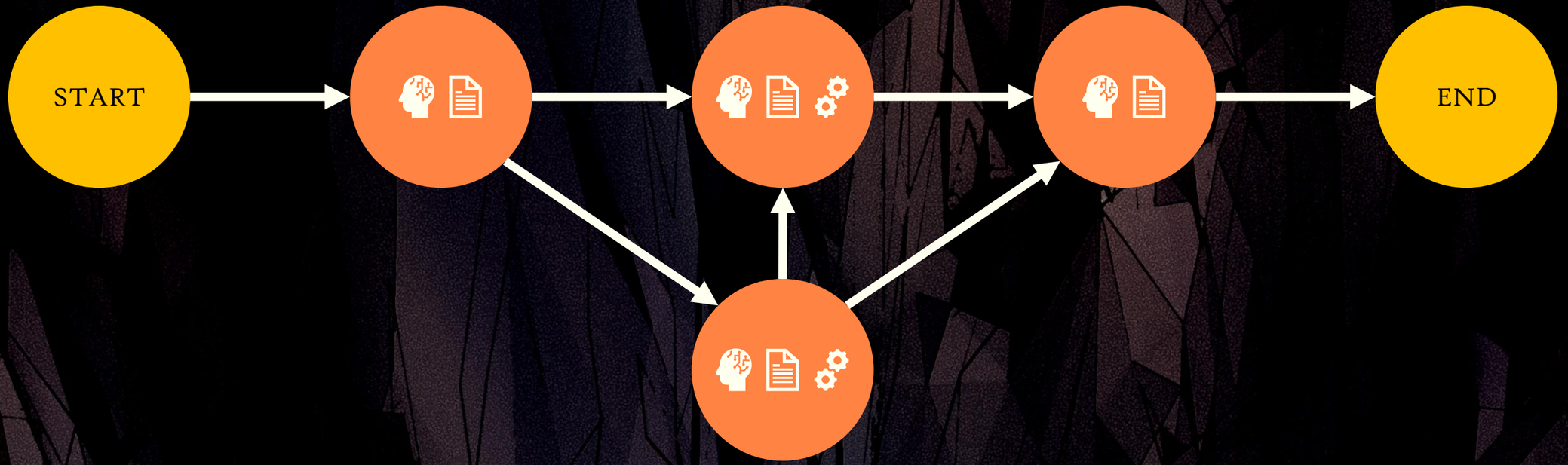


PROMPT

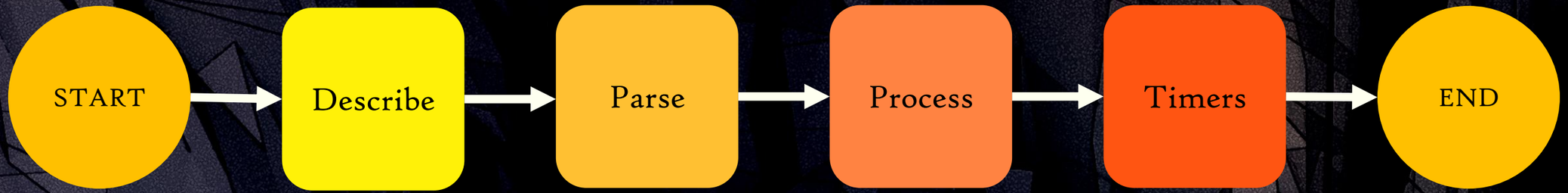


TOOLS

COMPOSING AGENTS



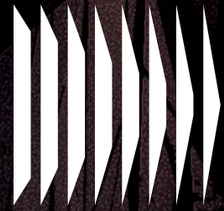
GAME LOOP



TOOL STACK









langchain

langgraph


langchain

SETTING UP THE LLM

```
const llm = new ChatOpenAI({
  model: 'gpt-6-luna',
  apiKey: OPENAI_API_KEY
})
```

MESSAGES

SystemMessage

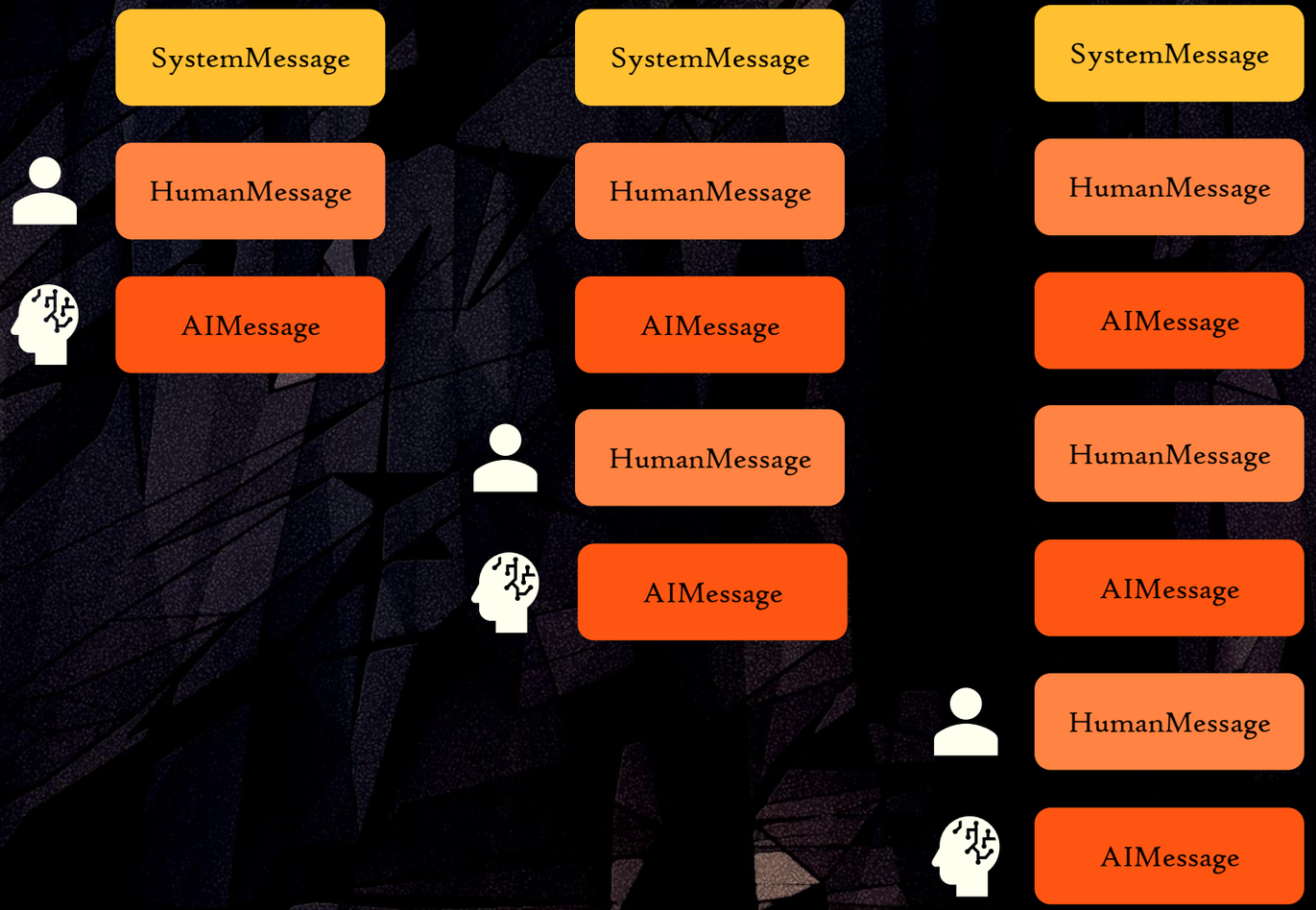
You are a helpful assistant that answers questions about Zork.

HumanMessage

What is Zork?

AIMessage

Zork is a text adventure game developed in the late 1970s at MIT.



CALLING THE LLM

```
const llm = new ChatOpenAI({
  model: 'gpt-6-luna',
  apiKey: OPENAI_API_KEY
})

const messages: BaseMessage[] = [
  new SystemMessage('You are a helpful assistant that answers questions about Zork.'),
  new HumanMessage('What is Zork?'),
  new AIMessage('Zork is a text adventure game developed in the late 1970s'),
  new HumanMessage('Who created Zork?')
]

const response: AIMessage = await llm.invoke(messages)

console.log(response.content)
```

CHAINING



CHAINING CALLS

```
const summarizer = new ChatOpenAI({
  model: 'gpt-6-luna',
  apiKey: OPENAI_API_KEY
})

const parser = new StringOutputParser()

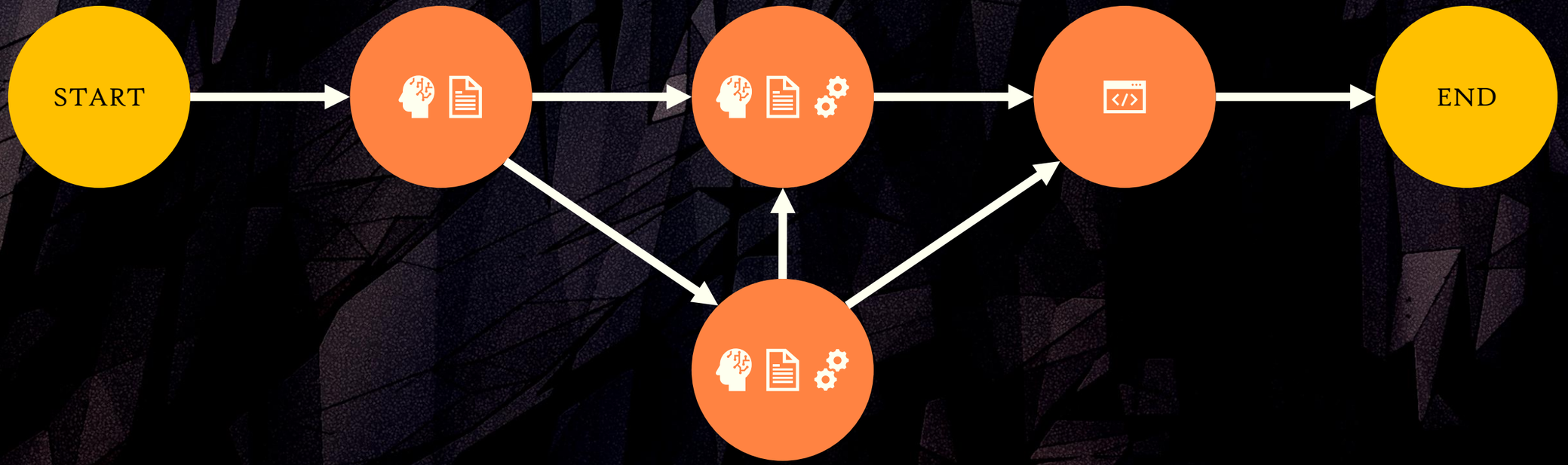
const embedder = new OpenAIEmbeddings({
  model: 'text-embedding-3-small',
  apiKey: OPENAI_API_KEY
})

const chain = PromptTemplate
  .fromTemplate('Summarize the following text in 2-3 sentences:\n\n{text}')
  .pipe(summarizer)
  .pipe(parser)
  .pipe(summary => embedder.embedQuery(summary))

const embedding = await chain.invoke({ text: 'Zork is a text adventure game...', })
```


langgraph

NODES IN A GRAPH



A GRAPH NEEDS STATE



Graph State

Key	Value
Key	Value
Key	Value

DEFINING NODES

Zork Expert

```
async function zorkExpertNode(state: typeof MessagesAnnotation.State) {

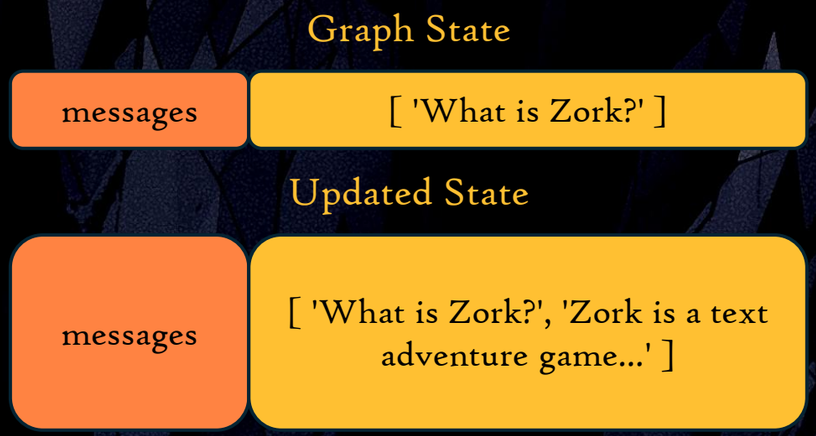
  const SYSTEM_PROMPT = dedent`
    You are a helpful assistant that answers questions
    about the classic text adventure game Zork.`

  const llm = fetchLLM()

  const response = await llm.invoke([
    new SystemMessage(SYSTEM_PROMPT),
    ...state.messages
  ])

  return {
    messages: [ response ]
  }

})
```



DEFINING GRAPH STATE

messages	[]

```ts
const MessagesAnnotation = Annotation.Root({

  messages: Annotation<BaseMessage[]>({
    default: () => [],
    reducer: (prev, next) => [
      ...prev,
      ...next
    ]
  })

})
```

question	'What is Zork?'
answer	'Zork is a text adventure game...'

```ts
const QNA_Annotation = Annotation.Root({

  question: Annotation<string>({
    default: () => '',
    reducer: (prev, next) => next
  }),

  answer: Annotation<string>({
    default: () => '',
    reducer: (prev, next) => next
  })

})
```

DEFINING A GRAPH




```
const graph = new StateGraph(MessagesAnnotation) as any

graph.addNode('zork_expert', zorkExpertNode)

graph.addEdge(START, 'zork_expert')
graph.addEdge('zork_expert', END)

const workflow = graph.compile()
```

INVOKING A GRAPH

What does the brass lantern do?



It keeps you from being eaten by a grue.

```
const graph = new StateGraph(MessagesAnnotation) as any

graph.addNode('zork_expert', zorkExpertNode)

graph.addEdge(START, 'zork_expert')
graph.addEdge('zork_expert', END)

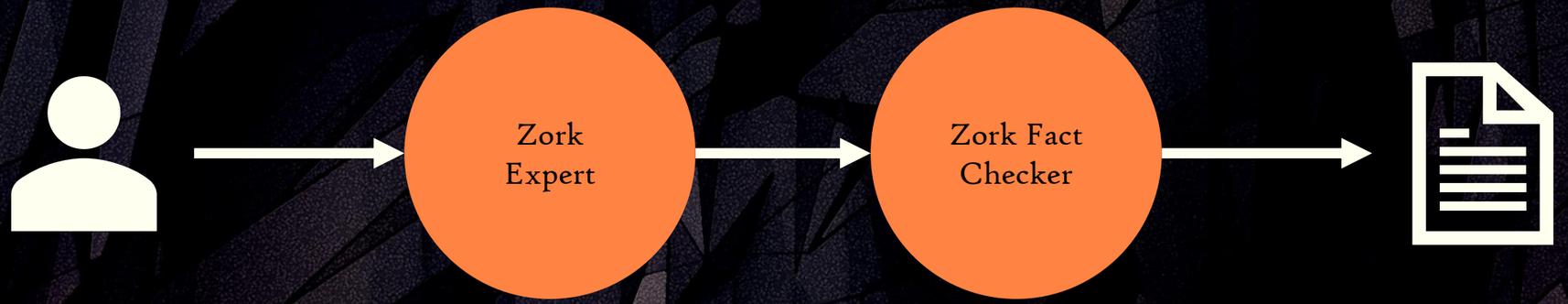
const workflow = graph.compile()

const initialState = {
  messages: [ new HumanMessage('What does the brass lantern do?') ]
}

const finalState = await workflow.invoke(initialState)
```

Initial State		Final State	
messages	['What does the brass lantern do? ']	messages	['What does the brass lantern do? ' , 'It keeps you from being eaten by a grue.]

CHAINED NODES

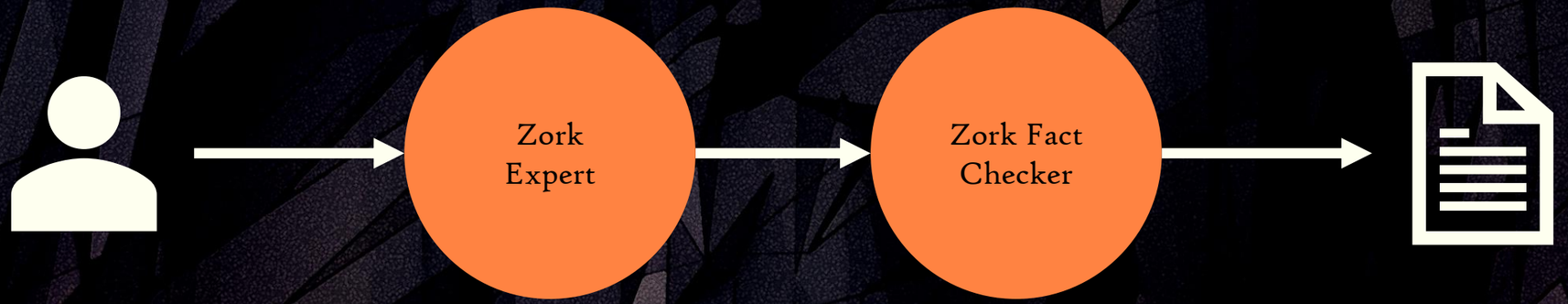


What is Zork?

Zork is the original text adventure game and was written in 1965 by Milton Bradley.

Zork is an early text adventure game and was developed in the late 1970s by Tim Anderson, Marc Blank, Bruce Daniels, and Dave Lebling at MIT.

CHAINED NODES



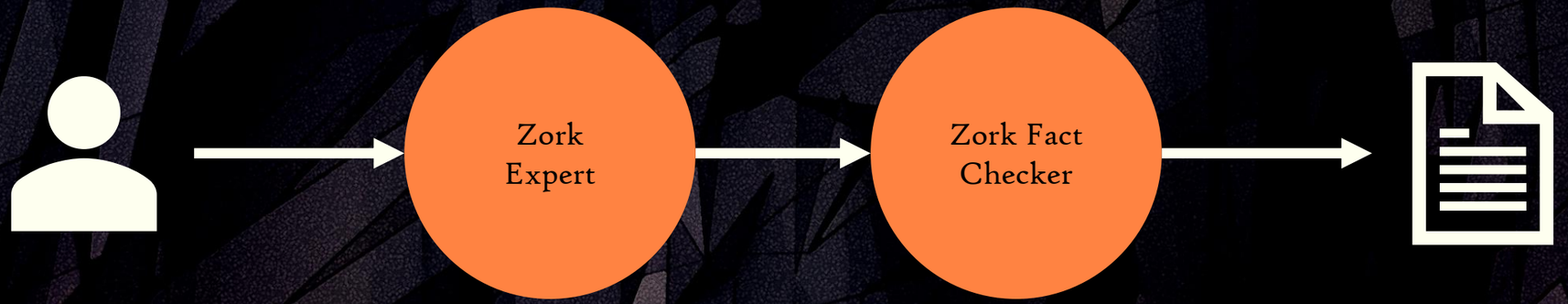

```
const graph = new StateGraph(MessagesAnnotation) as any

graph.addNode('zork_expert', zorkExpertNode)
graph.addNode('zork_fact_checker', zorkFactCheckerNode)

graph.addEdge(START, 'zork_expert')
graph.addEdge('zork_expert', 'zork_fact_checker')
graph.addEdge('zork_fact_checker', END)

const workflow = graph.compile()
```

STATE IN CHAINED NODES



What is Zork?

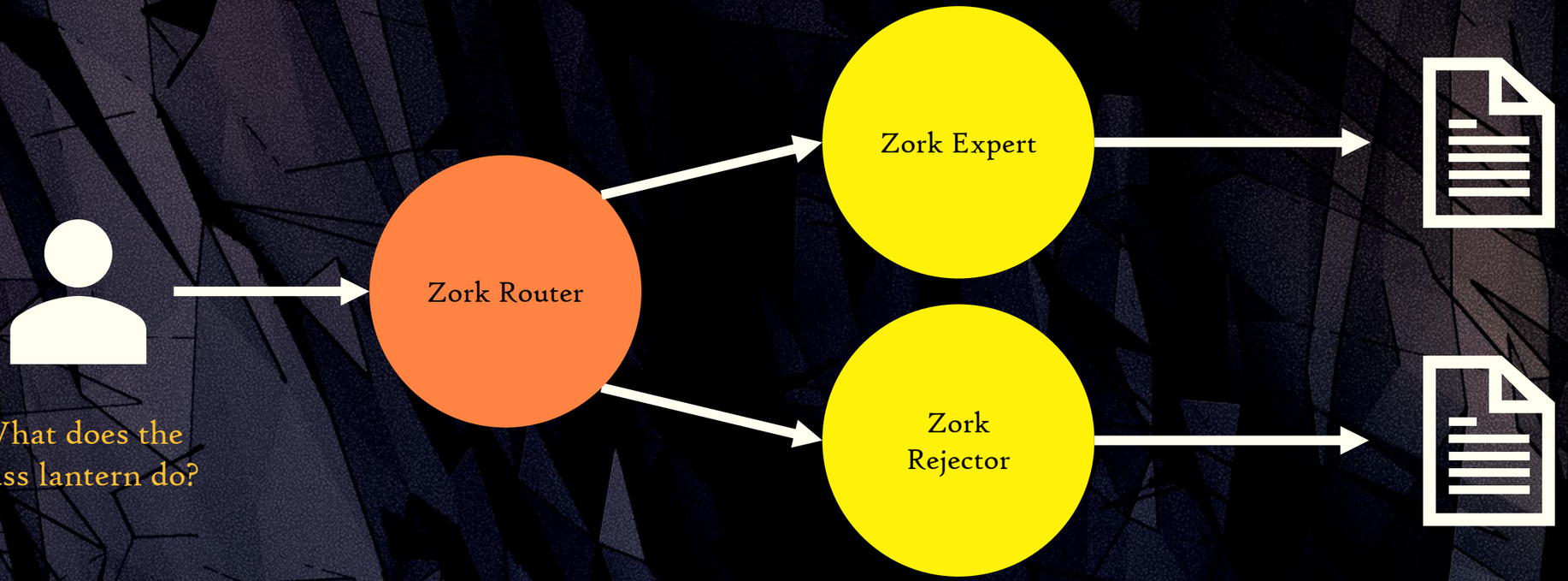
Zork is the original text adventure game...

Zork is an early text adventure game...

messages

```
[
    'What is Zork?',
    'Zork is the original text adventure game...'
    'Zork is an early text adventure game...'
]
```

ROUTING



What does the brass lantern do?

It keeps you from being eaten by a grue.

I'm sorry, I can only answer questions about Zork.

```
graph.addEdge(START, 'zork_router')
graph.addConditionalEdges('zork_router', routeByTopic)
graph.addEdge('zork_expert', END)
graph.addEdge('zork_rejector', END)
```

```
function routeByTopic(state) {
  const lastMessage = state.messages.at(-1)
  const content = lastMessage.content
  return content === 'zork' ? 'zork_expert' : 'zork_rejector'
}
```

THE ROUTER

```
async function zorkRouter(state: typeof MessagesAnnotation.State) {

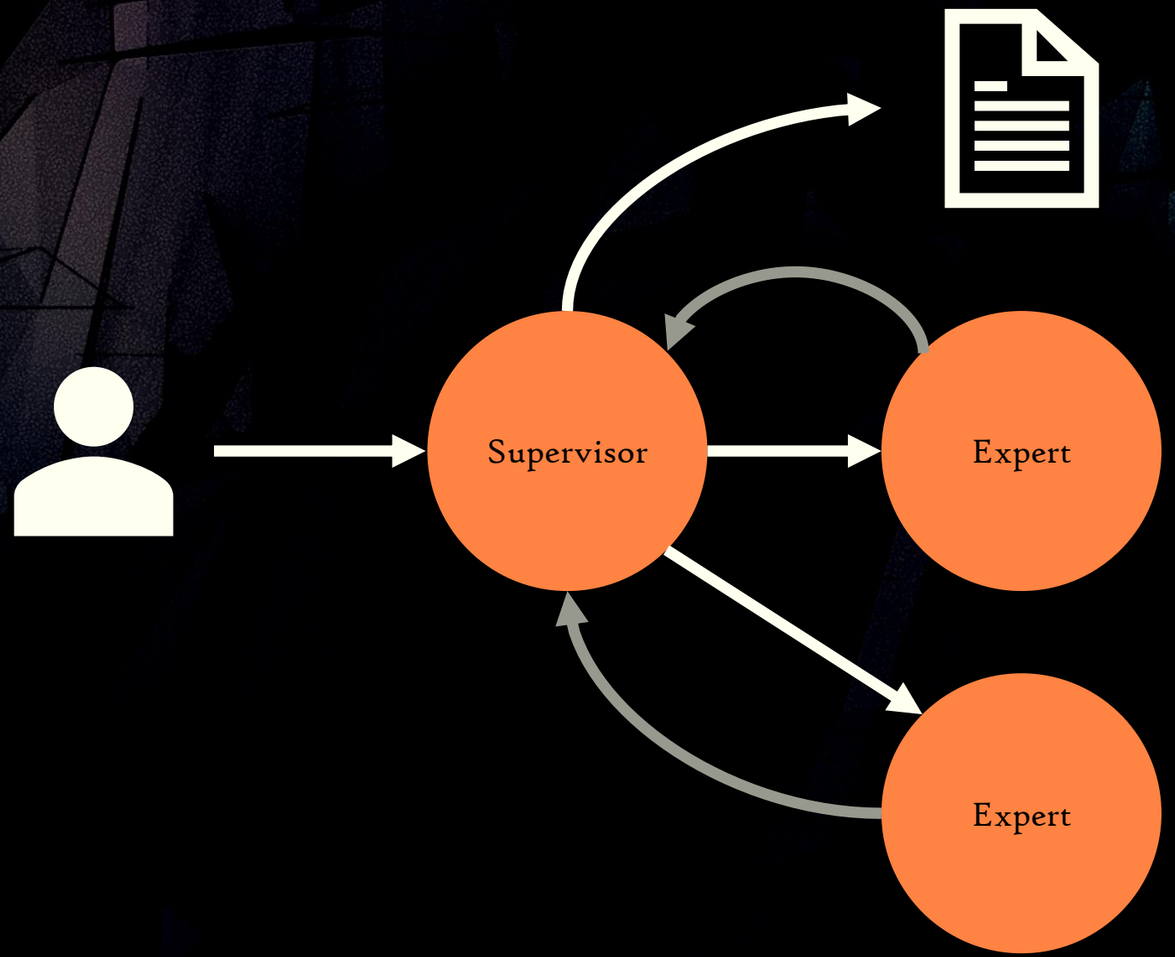
  const SYSTEM_PROMPT = dedent`
    You are a helpful assistant that routes questions to the right expert. You will be
    provided with a question and you should decide if it is related to Zork. If it is
    related to Zork, you should respond with "zork". If it is not related to Zork, you
    should respond with "other".`

  const systemMessage = new SystemMessage(SYSTEM_PROMPT)
  const receivedMessages: BaseMessage[] = state.messages

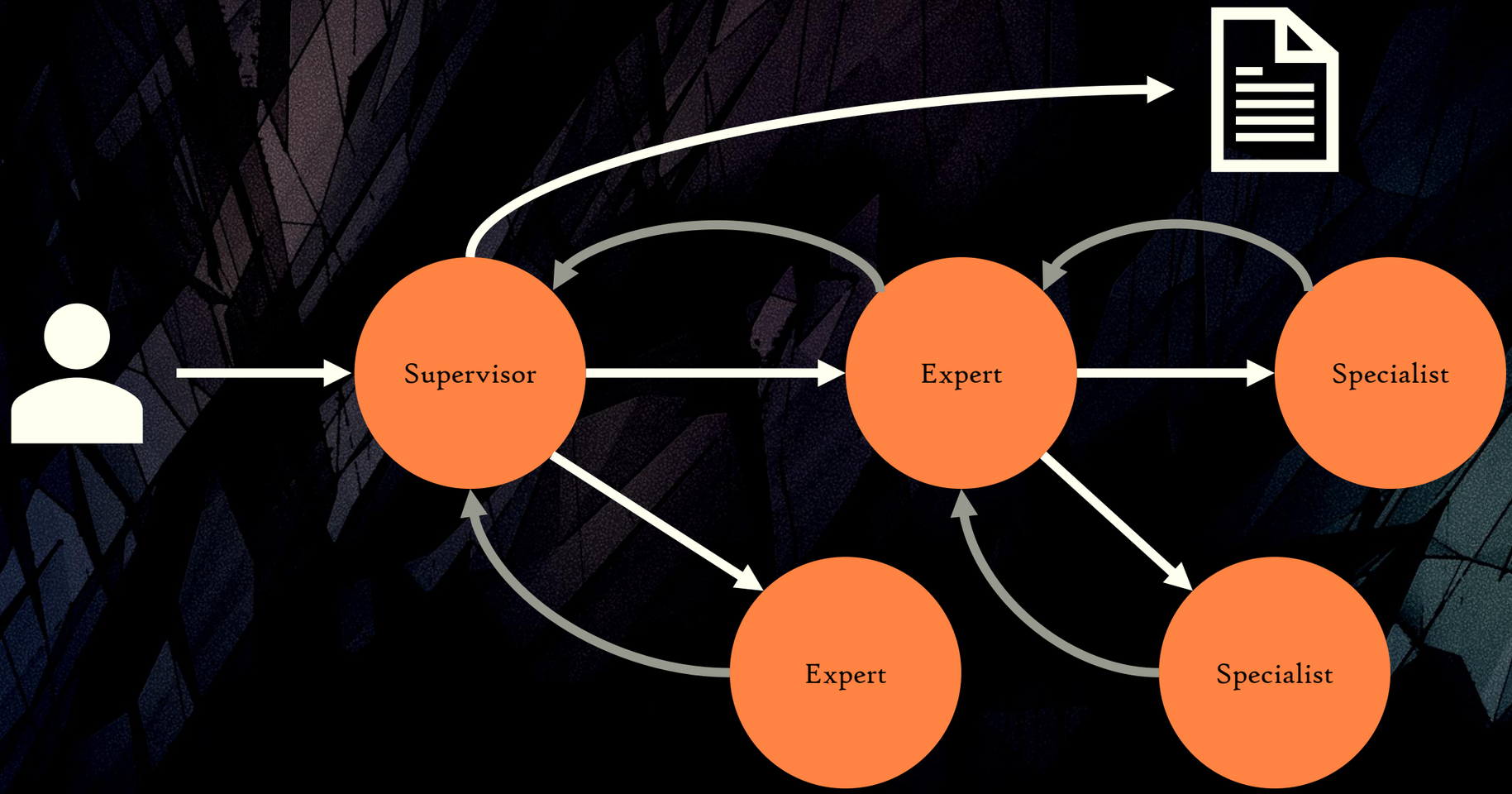
  const response: AIMessage = await llm.invoke([systemMessage, ...receivedMessages])

  return { messages: [response] }
}
```

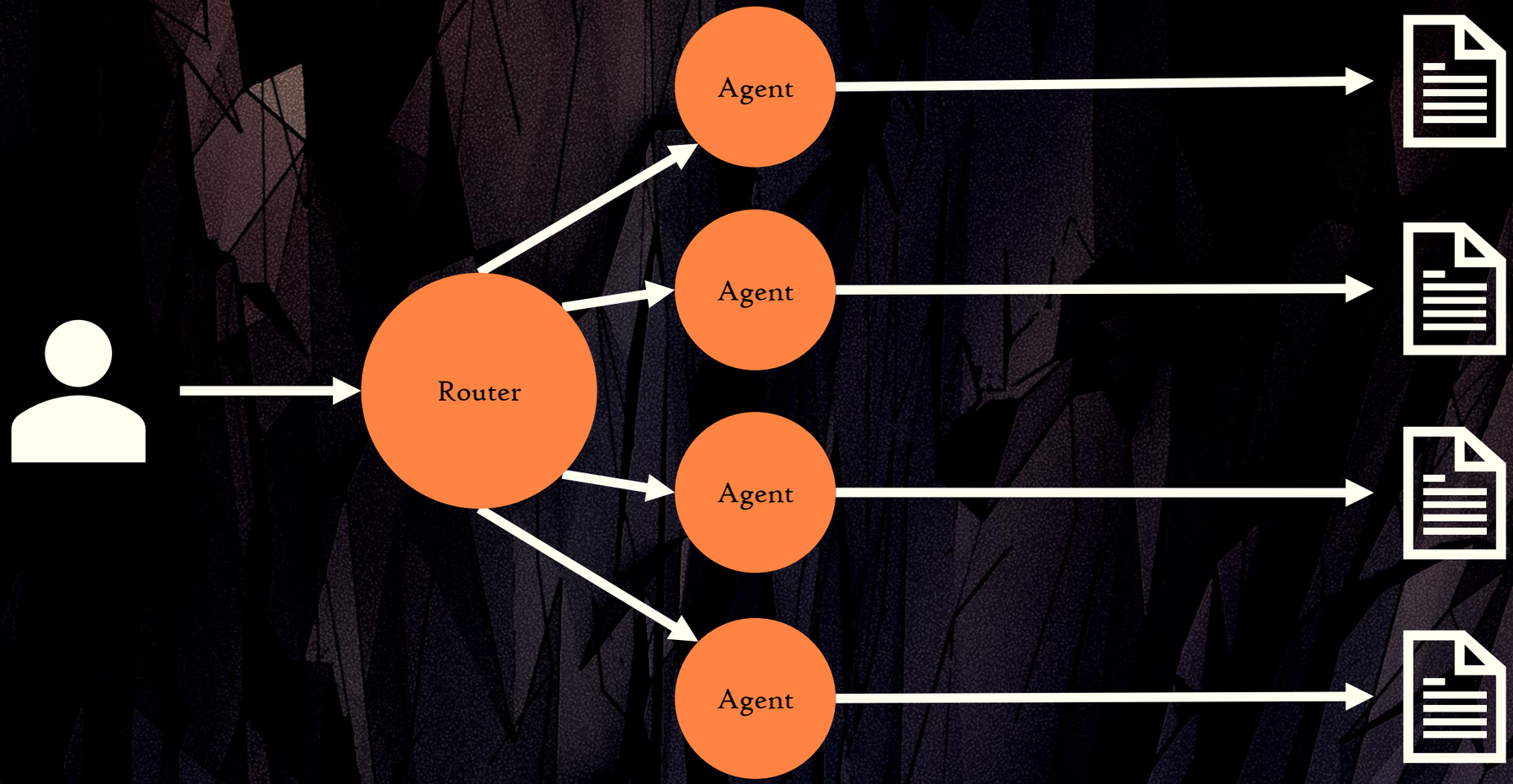
SUPERVISOR



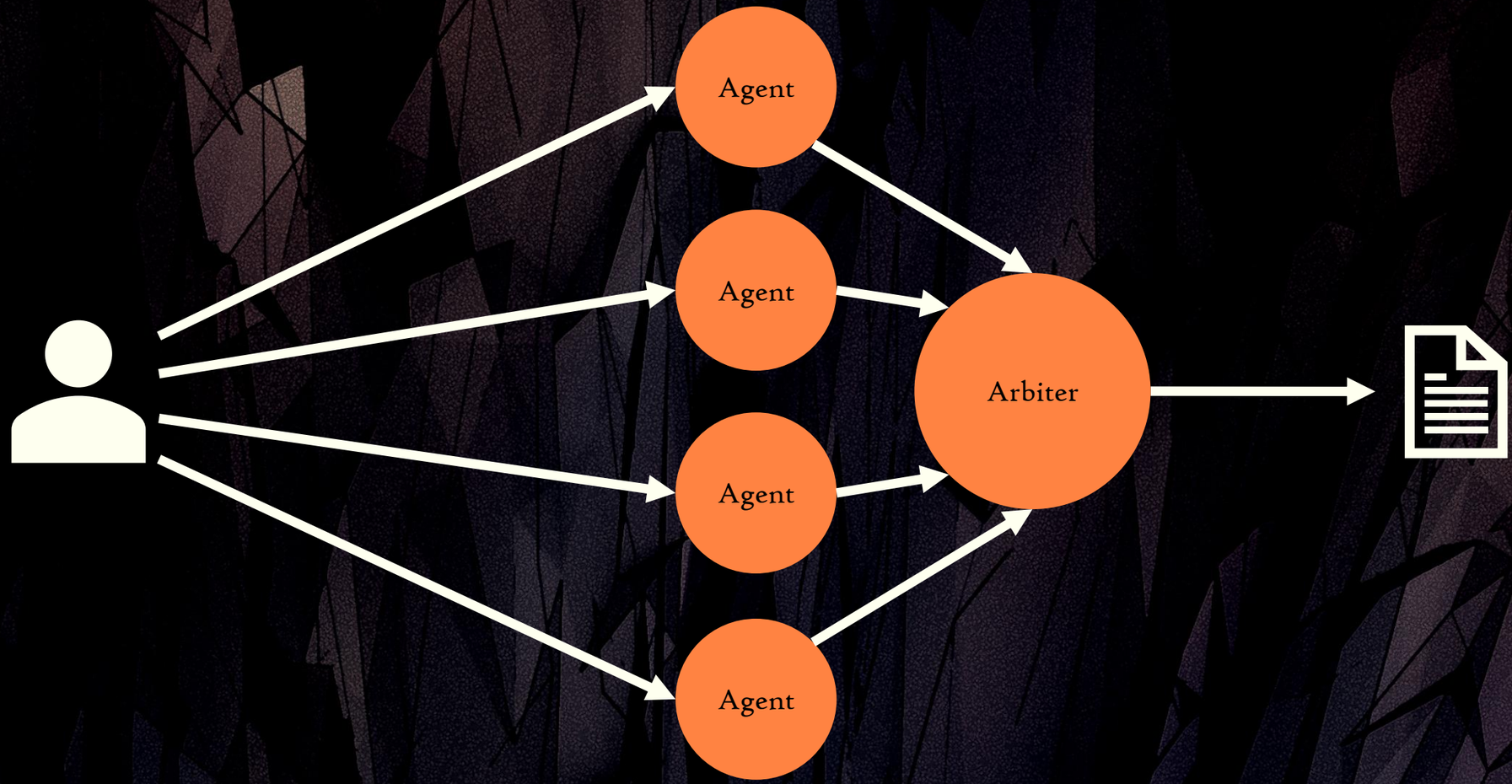
HIERARCHICAL



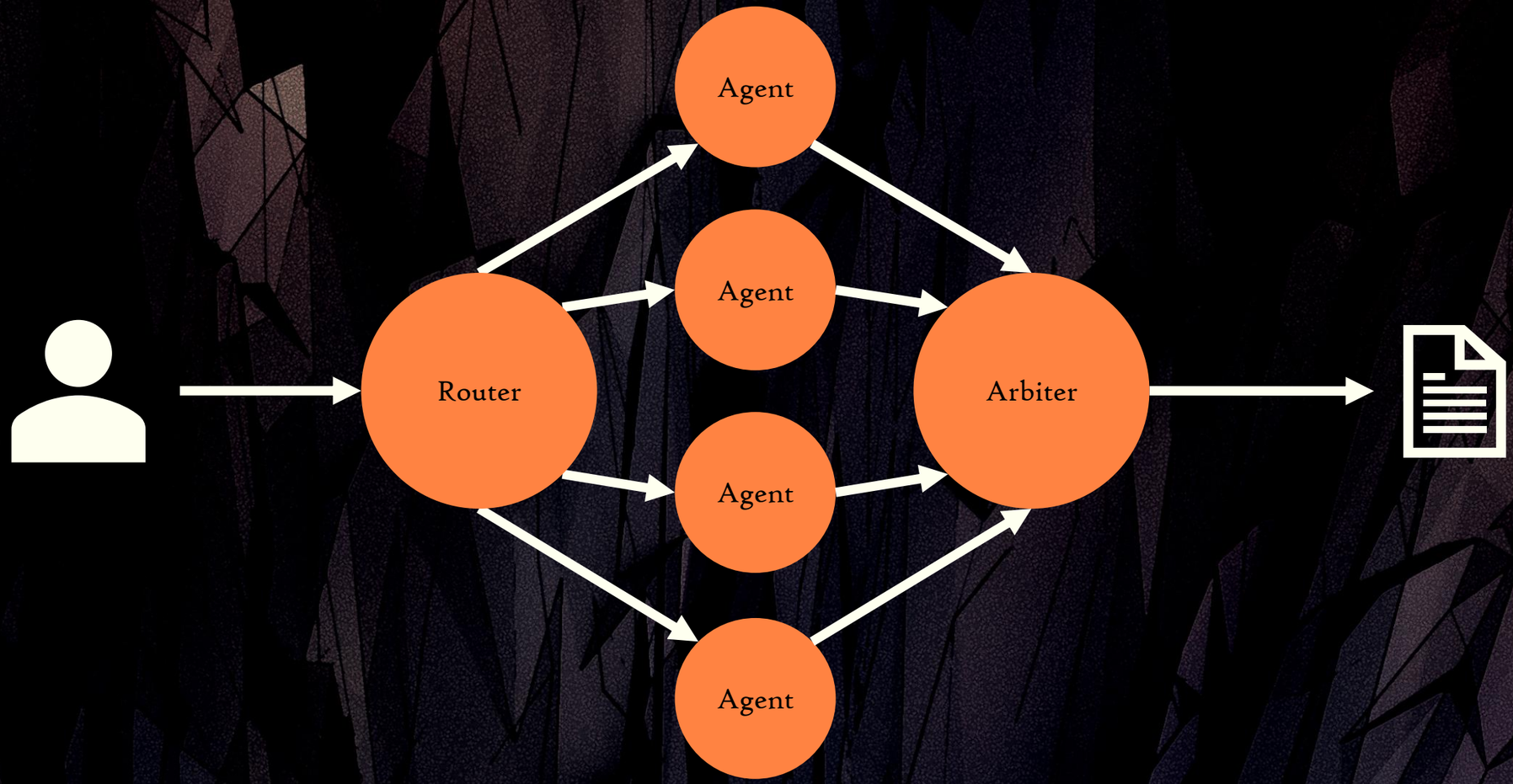
FAN OUT



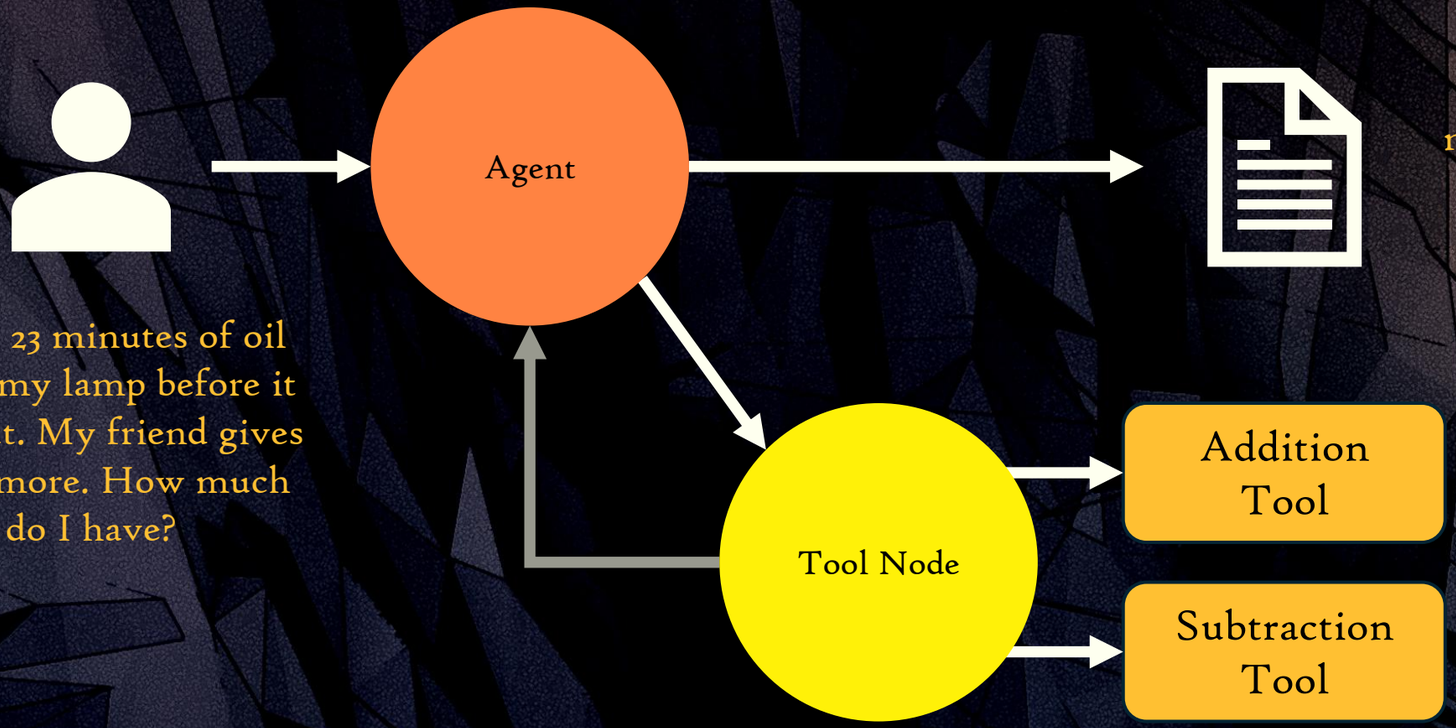
FAN IN



FAN IN AND OUT



TOOL USE


I have 23 minutes of oil left in my lamp before it goes out. My friend gives me 17 more. How much do I have?
Agent
Tool Node
Addition Tool
Subtraction Tool
You have a total of 40 minutes left before your lamp goes out and you are eaten by a grue.

DEFINING TOOLS

```
const additionTool = tool(
  args => args.numbers.reduce((a, b) => a + b),
  {
    name: 'AddArrayOfNumbers',
    description: 'Accepts an array of numbers and adds them together',
    schema: z.object({
      numbers: z.array(z.number()).describe('The numbers to add together') })
  }
)

const subtractionTool = tool(
  args => args.numbers.reduce((a, b) => a - b),
  {
    name: 'SubtractArrayOfNumbers',
    description: 'Accepts an array of numbers and subtracts them all from the first one',
    schema: z.object({
      numbers: z.array(z.number()).describe('The numbers to subtract') })
  }
)
```

USING REACT AGENTS AS NODES

```
const reactAgent = createReactAgent({
  llm: fetchLLM(),
  [ additionTool, subtractionTool ],
  messageModifier: new SystemMessage(SYSTEM_PROMPT)
})

const graph = new StateGraph(MessagesAnnotation) as any

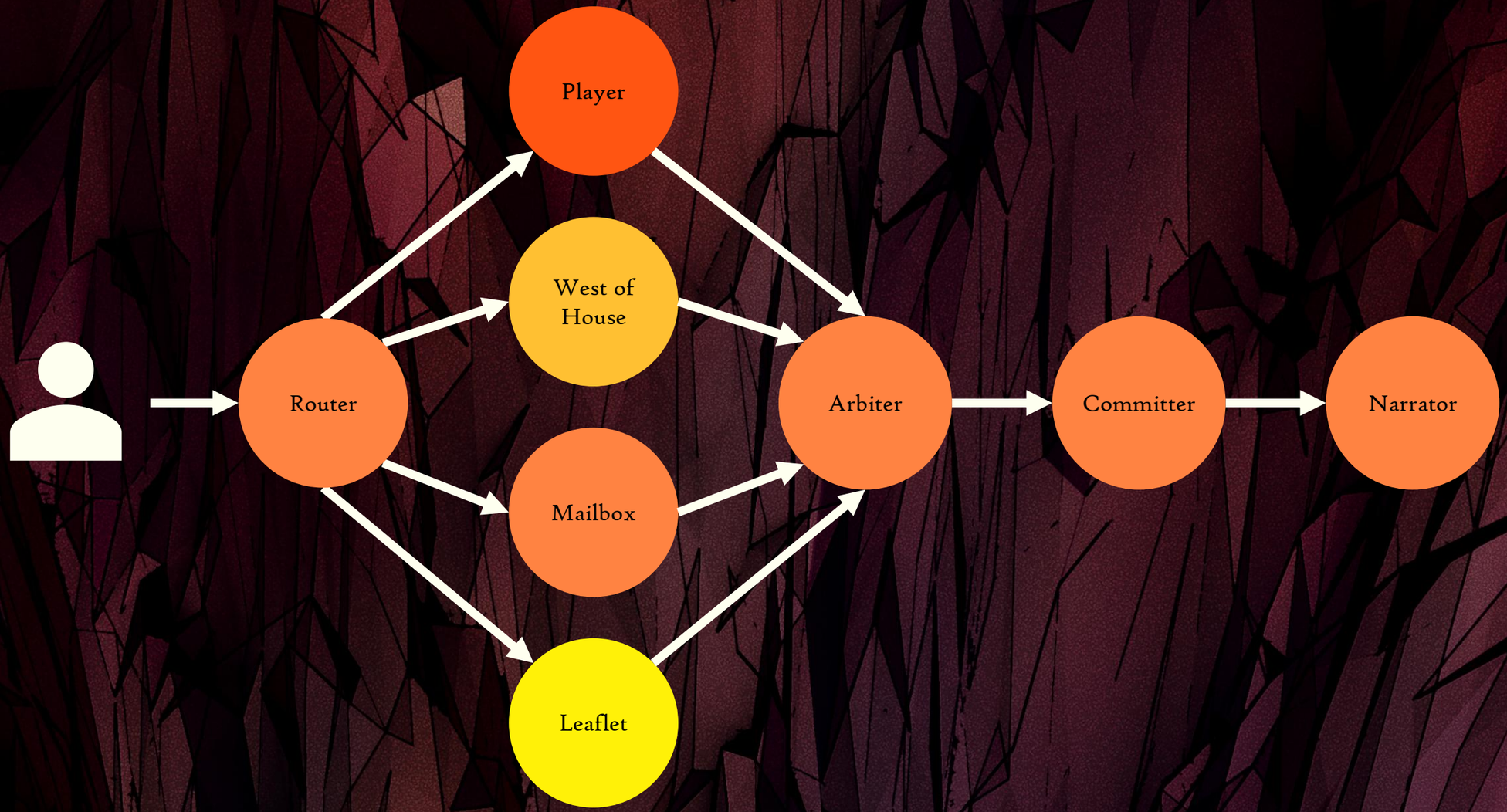
graph.addNode('zork_calculator', reactAgent)

graph.addEdge(START, 'zork_ calculator ')
graph.addEdge('zork_ calculator ', END)

const workflow = graph.compile()
```

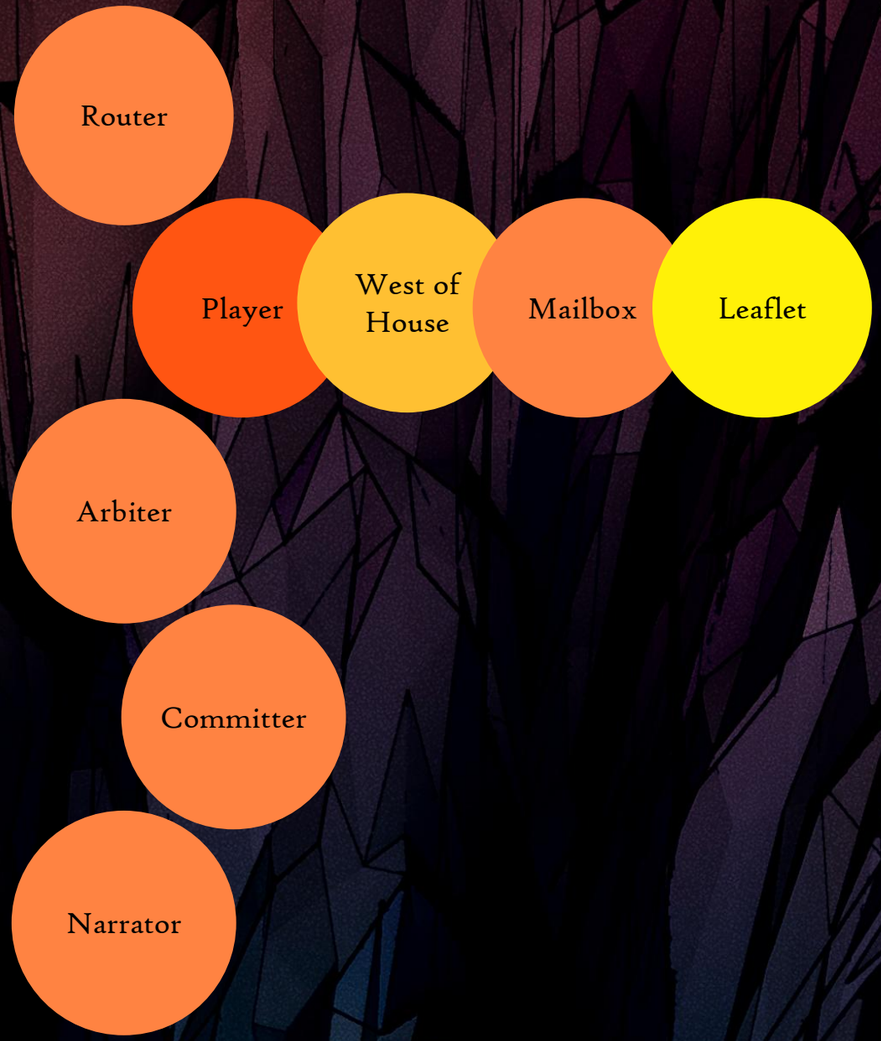

DOOR

GRAPH FOR RETURN TO ZORK



RETURN TO ZORK GRAPH STATE





```ts
export const GameTurnAnnotation = Annotation.Root({
  userCommand: Annotation<string | null>({
    reducer: (prev, next) => next,
    default: () => null
  }),

  selectedEntities: Annotation<SelectedEntity[]>({
    reducer: (prev, next) => next,
    default: () => []
  }),

  entityChangeRecommendations: Annotation<EntityChangeRecommendation[] | EntityChangeRecommendation>({
    reducer: (prev, next) => {
      const result = []
      Array.isArray(prev) ? result.push(...prev) : result.push(prev)
      Array.isArray(next) ? result.push(...next) : result.push(next)
      return result
    },
    default: () => []
  }),

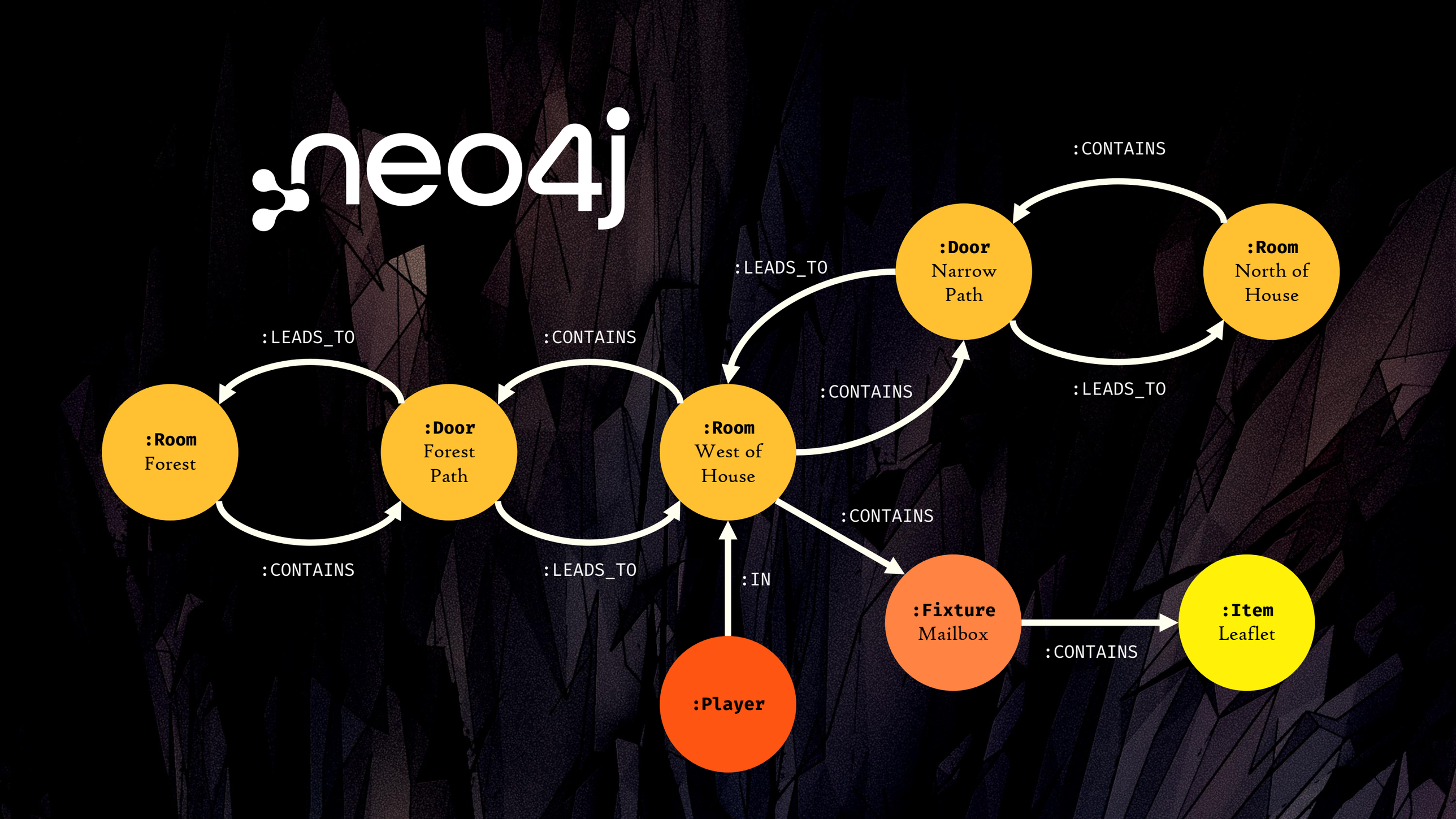
  entityChanges: Annotation<EntityChange[]>({
    reducer: (_prev, next) => next,
    default: () => []
  }),

  finalNarrative: Annotation<string | null>({
    reducer: (_prev, next) => next,
    default: () => null
  })
})
```


MEMORY

GAME STATE


North of House
"This is a narrow field with woods to the north and a white house to the south. There is no door and all the windows are boarded up. "
"This is an open and grassy field. There is a white house to the east with a boarded front door."
"This forest has trees in all directions."
Forest
West of House
Mailbox
"Rusted Shut", "Leaning"
Player
Leaflet
"Folded Neatly"

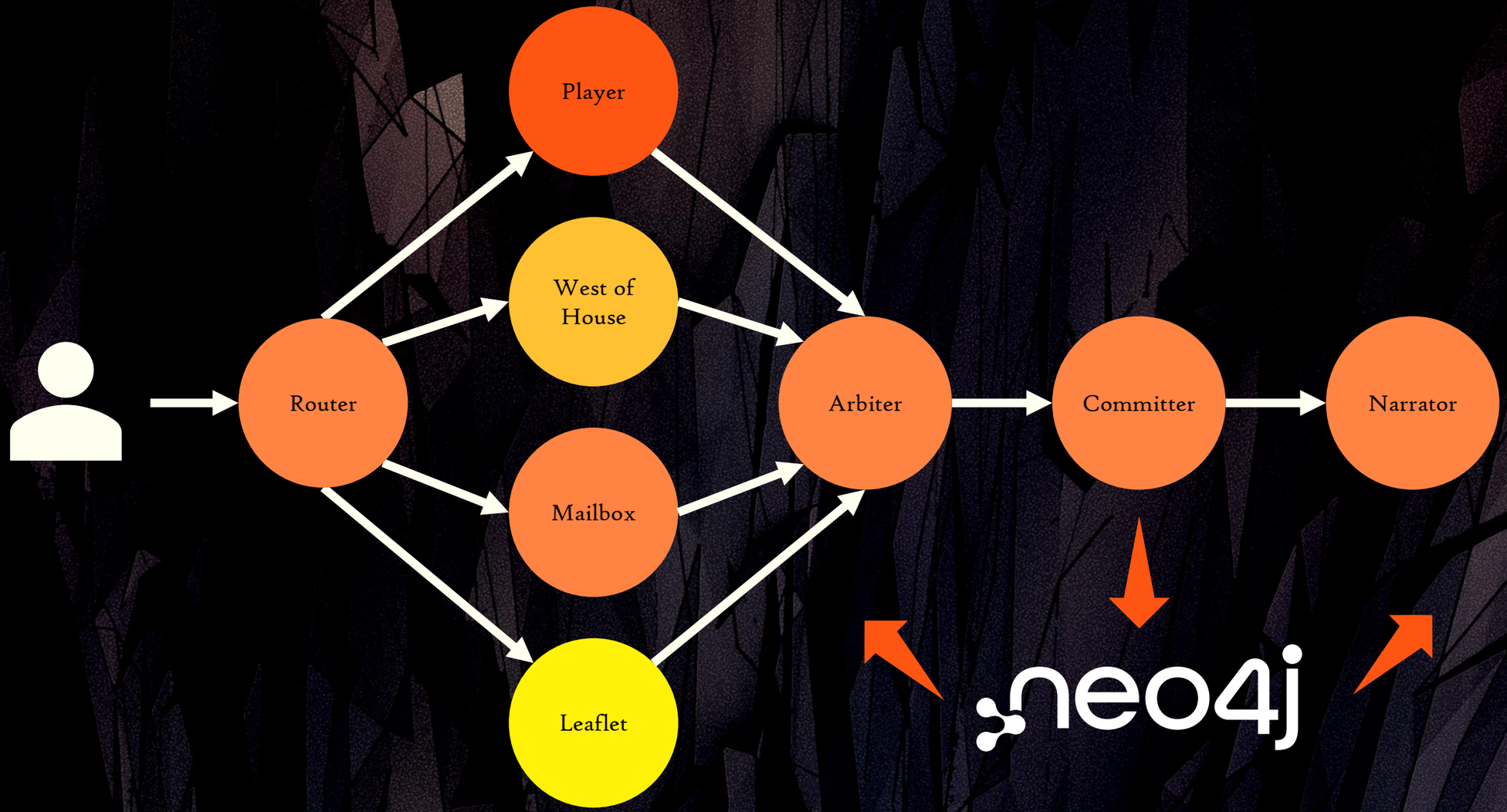

neo4j
:Room
Forest
:Door
Forest
Path
:Room
West of
House
:Door
Narrow
Path
:Room
North of
House
:Fixture
Mailbox
:Item
Leaflet
:Player
:LEADS_TO
:CONTAINS
:LEADS_TO
:CONTAINS
:CONTAINS
:LEADS_TO
:CONTAINS
:LEADS_TO
:CONTAINS
:IN
:CONTAINS

NEO4J FROM TYPESCRIPT

```
async #getRoom(): Promise<RoomEntity | null> {
  const data = await runOne<RoomData>`
    MATCH (player:Player) WHERE player.instanceId = ${this.instanceId}
    MATCH (player)-[:IN]->(room:Room)
    RETURN room {
      .instanceId,
      .entityId,
      .name,
      .description,
      .statuses
    } AS room
  `

  return data ? new RoomEntity(data) : null
}
```

ACCESSING GAME STATE



GAME STATE ISN'T ENOUGH



You are West of the House. There is a rusty mailbox on a post leaning at an angle.



I open the mailbox.



You yank on the mailbox and manage to break through years of rust. The mailbox is now open.

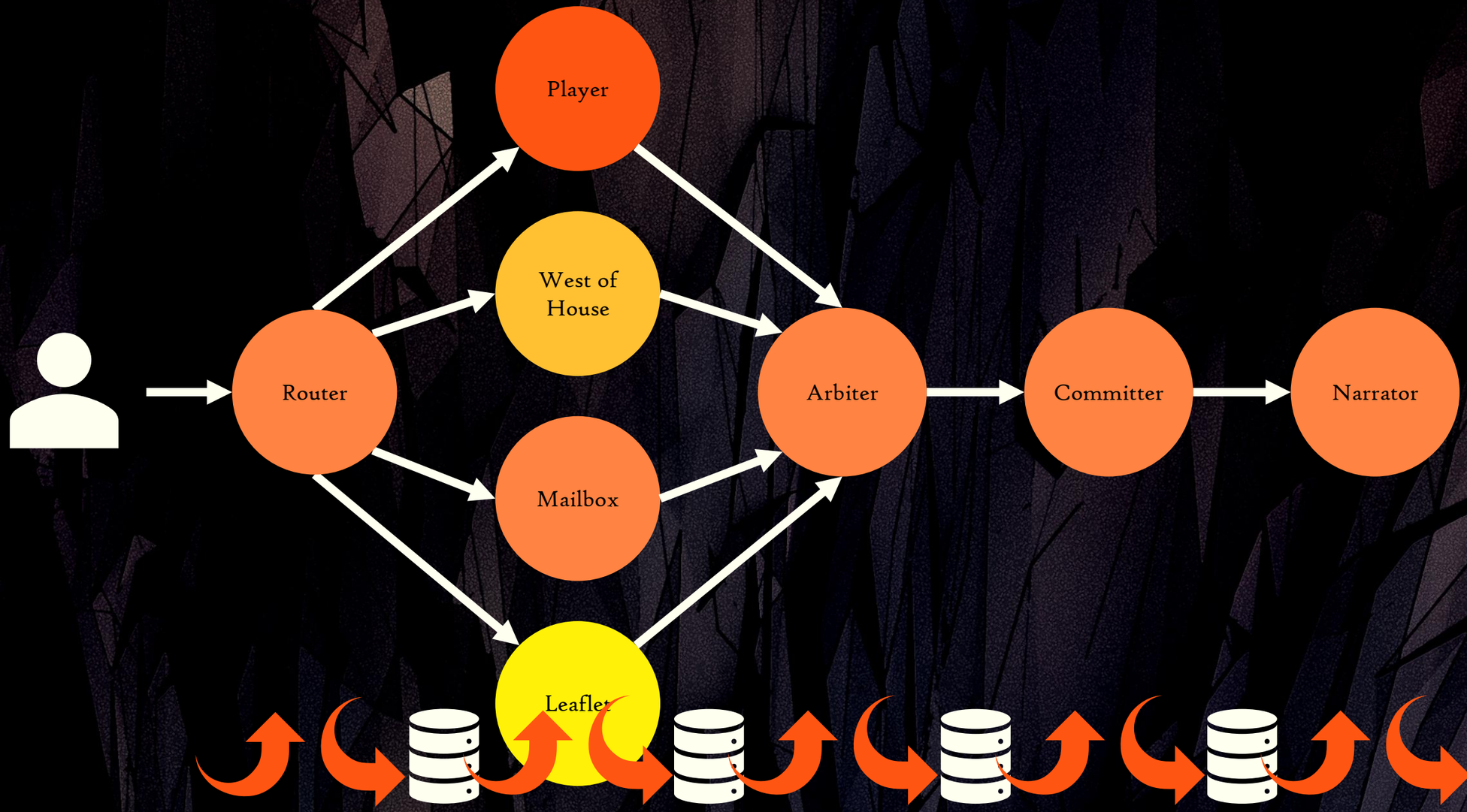


Is there anything in it?

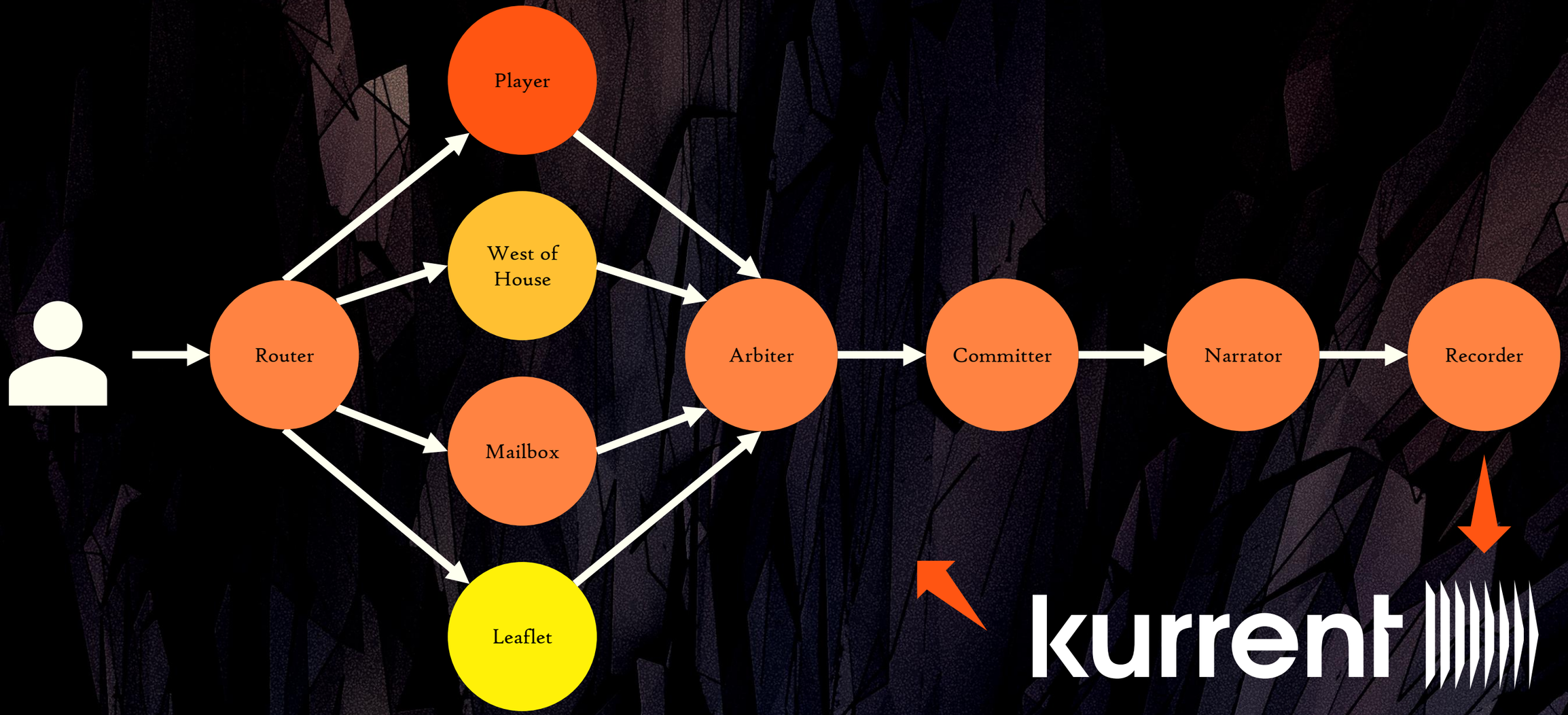


In what?

CHECKPOINTER SAVES GRAPH STATE



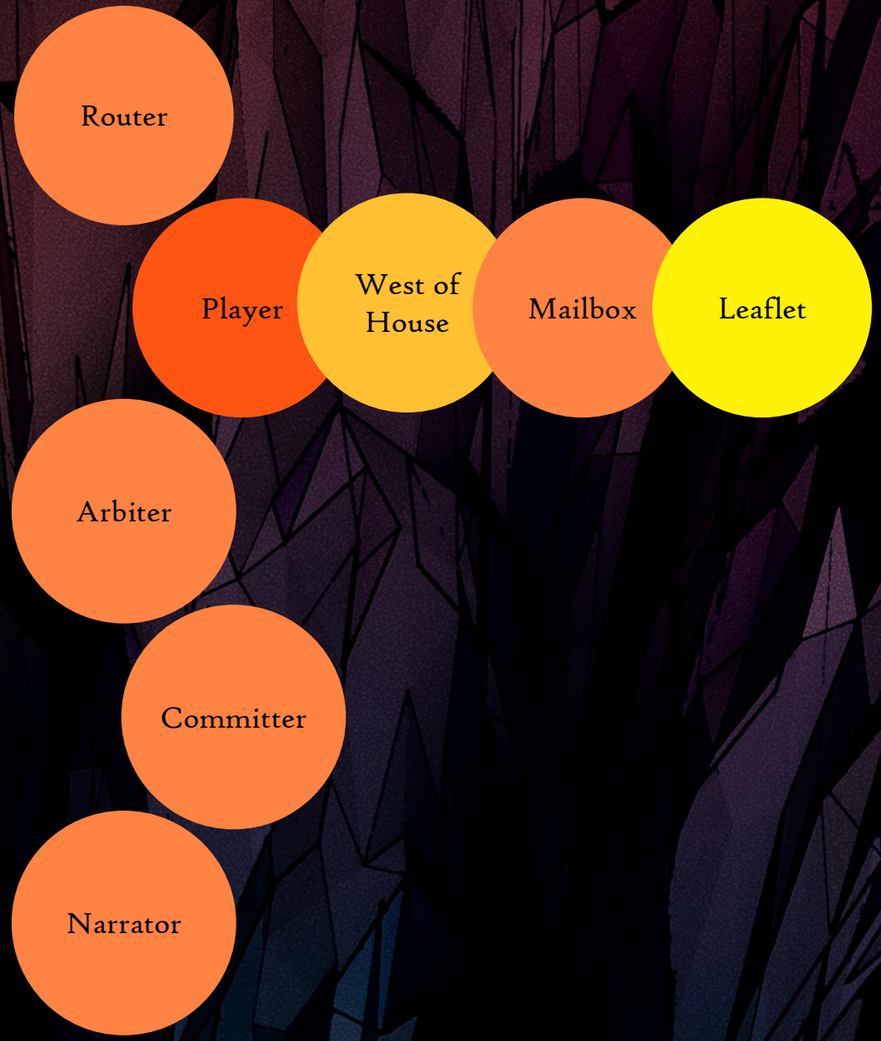
SAVE A LITTLE HISTORY



kurrent

RETURN TO ZORK GRAPH STATE



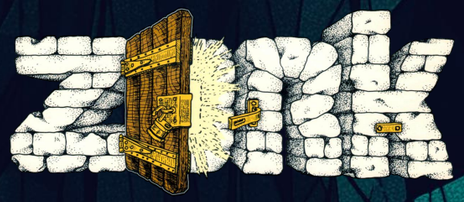


DEMO

THE GUY WIRE

medium.com/the-guy-wire

RESOURCES



github.com/historicalsource/zork1
github.com/historicalsource/zork2
github.com/historicalsource/zork3
davidgriffith.gitlab.io/frotz/

LangChain

docs.langchain.com/oss/javascript/langchain/overview
docs.langchain.com/oss/javascript/langgraph/overview

neo4j

neo4j.com/docs/getting-started/

kurrent

github.com/kurrent-io/KurrentDB

bitly

github.com/guyroyse/
return-to-zork



Guy Royse

youtube.com/@guyroyse

github.com/guyroyse

@guy.dev

@guyroyse

guy.dev


THANKS